



**Politecnico
di Torino**

Applied Data Science Project

Transfer Learning with VLMs

FEDERICO D'ASARO

APPLIED DATA SCIENCE PROJECT, A.Y. 2025/2026

What is Transfer Learning

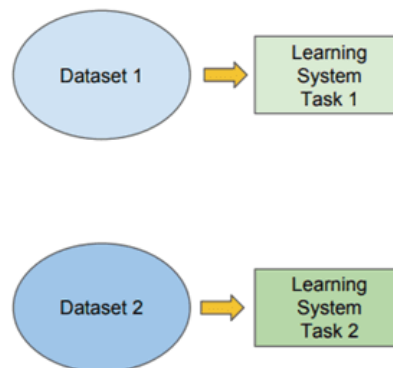
Definition: Transfer learning is the technique of reusing a model trained on one task as the starting point for a different but related task.

Why it matters:

- Reduces the need for large labeled datasets by reusing knowledge from pre-trained models.
- Speeds up training and improves performance.

Traditional ML

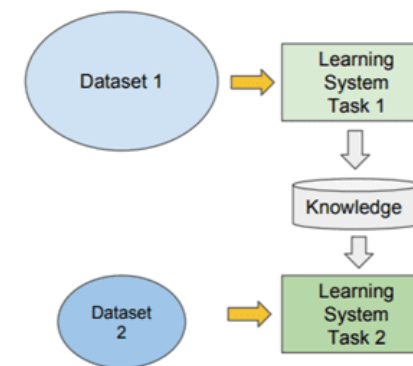
- Isolated, single task learning:
 - Knowledge is not retained or accumulated. Learning is performed w.o. considering past learned knowledge in other tasks



vs

Transfer Learning

- Learning of a new task relies on the previous learned tasks:
 - Learning process can be faster, more accurate and/or need less training data



Initial Approach to Transfer Learning

- Start with a model pre-trained on a large source dataset.
- Fine-tune all network parameters on the target task.

Advantages

- The model adapts fully to the new domain.
- Can achieve high accuracy when enough labeled data is available.

Limitations

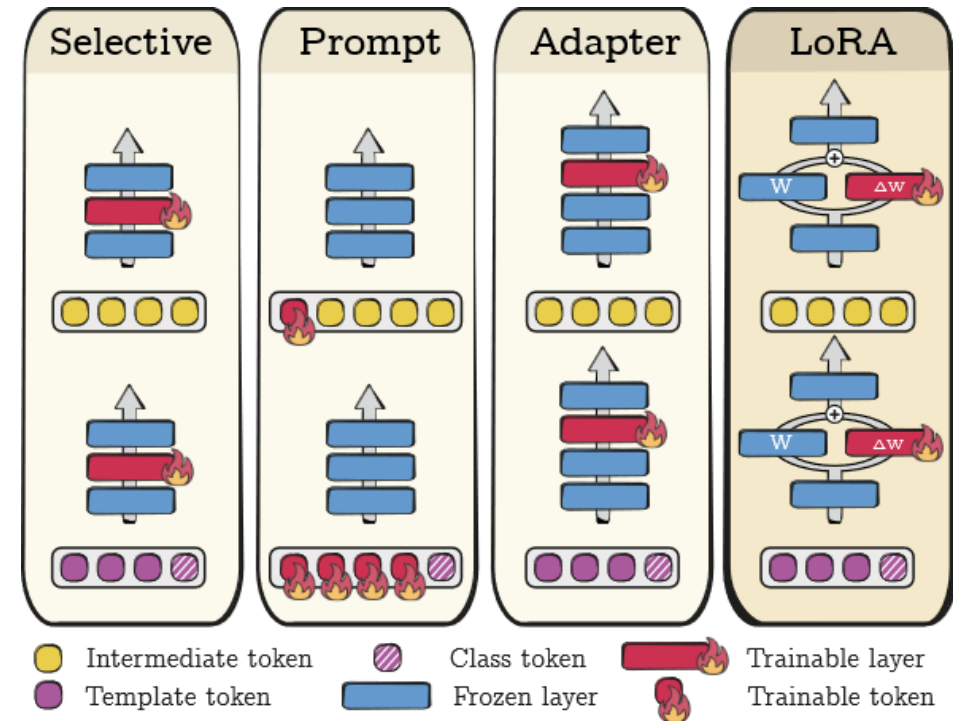
- Requires substantial computational resources.
- Risk of overfitting if the target dataset is small.
- Large pre-trained models (foundation models) may lose their generalization ability.
- Training can be time-consuming.

Efficient Approach Transfer Learning

Parameter-Efficient Fine-Tuning (PEFT): PEFT (Parameter-Efficient Fine-Tuning) adapts large Pre-trained Models by training only a small set of task-specific parameters while keeping the backbone frozen, reducing computation and memory cost.

PEFT techniques can be grouped into four categories:

- **Selective:** fine-tune only a subset of existing model weights.
- **Prompt Tuning:** add learnable tokens to the input or intermediate layers.
- **Adapters:** insert trainable modules into the frozen architecture; main drawback: extra parameters can increase inference latency.
- **Low-Rank Adaptation (LoRA):** decompose weight updates into low-rank matrices while keeping original parameters frozen.



Parameter-Efficient Fine-Tuning: Key Benefits

- **Increased Efficiency** – Adjusts only a small subset of parameters, reducing GPU, energy, and cloud costs.
- **Faster Time-to-Value** – Speeds up model updates for new tasks with minimal training overhead.
- **No Catastrophic Forgetting** – Retains original model knowledge while adapting to new data.
- **Lower Overfitting Risk** – Keeps most parameters fixed, improving generalization.
- **Reduced Data Needs** – Requires smaller datasets compared to full fine-tuning.
- **More Accessible AI** – Lowers cost barriers, enabling smaller teams to fine-tune LLMs.
- **Greater Flexibility** – Simplifies customization for domain-specific or specialized use cases.

[IBM: What is parameter-efficient fine-tuning \(PEFT\)?](#)

Selective Fine-Tuning

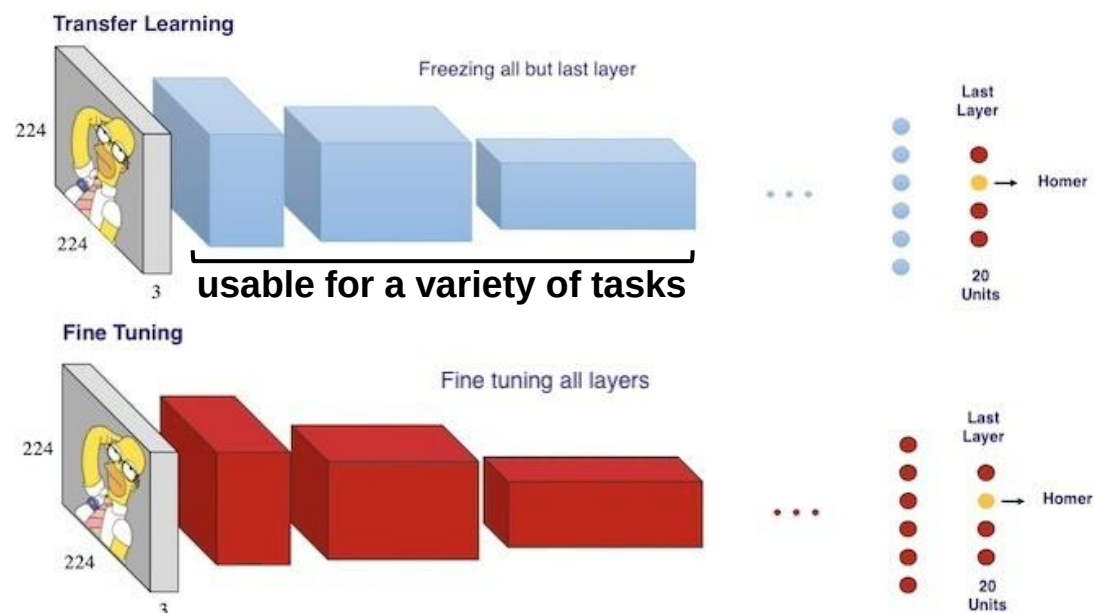
Method: Freeze backbone, fine-tune only the final layer.

Training Benefits

- Faster and cheaper training.
- Less risk of overfitting.

Deployment Benefits

- The shared backbone can be deployed once for multiple tasks.
- Only lightweight task-specific heads (last-layer parameters) need to be instantiated per task.
- Reduces memory and storage requirements.



Soft Prompt Tuning

Soft Prompt Tuning learns and optimizes text prompt tokens, which can also be extended to intermediate layers. In transformer-based architectures, *similar learnable tokens can be introduced into vision models*.

Input-level prompts (first layer): For class k , the input is augmented with M trainable prompt tokens and a fixed class token:

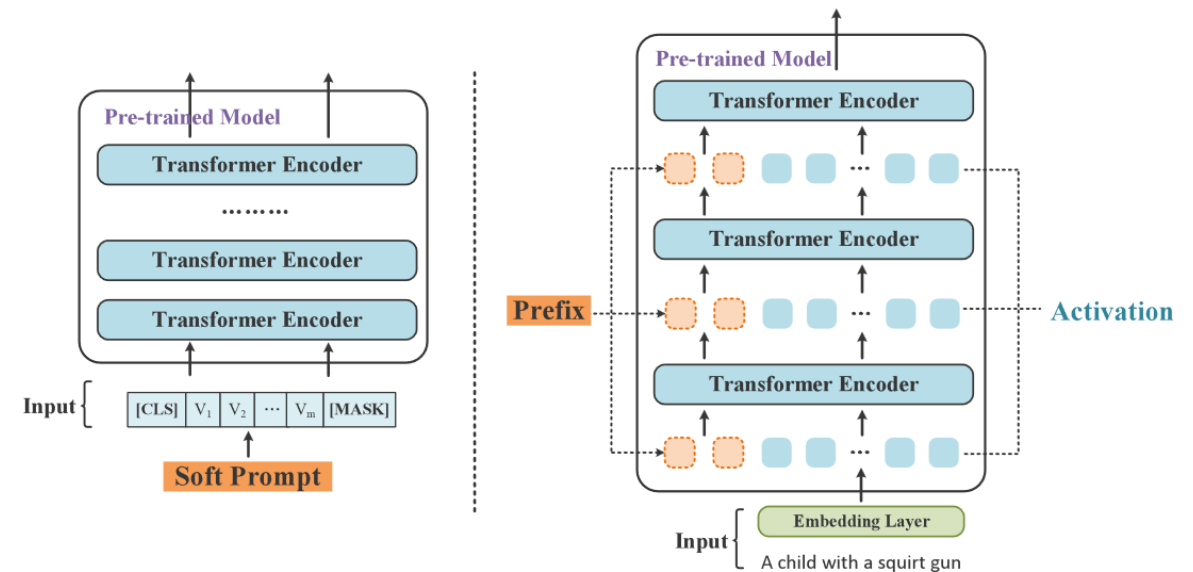
$$c_k = (v_1^k, \dots, v_M^k, [\text{class}_k])$$

where v_i^k are learnable prompt tokens and $[\text{class}_k]$ is the word embedding of the class name.

Intermediate-layer prompts: At transformer layer l , soft prompts are prepended to the sequence:

$$X^{(l)} = (v_1^{(l)}, \dots, v_{N_V}^{(l)}, x_1^{(l)}, \dots, x_{N_X}^{(l)})$$

where $v_i^{(l)}$ are learnable prompt tokens, and $x_i^{(l)}$ are the original input tokens.



Soft Prompt Tuning - Examples

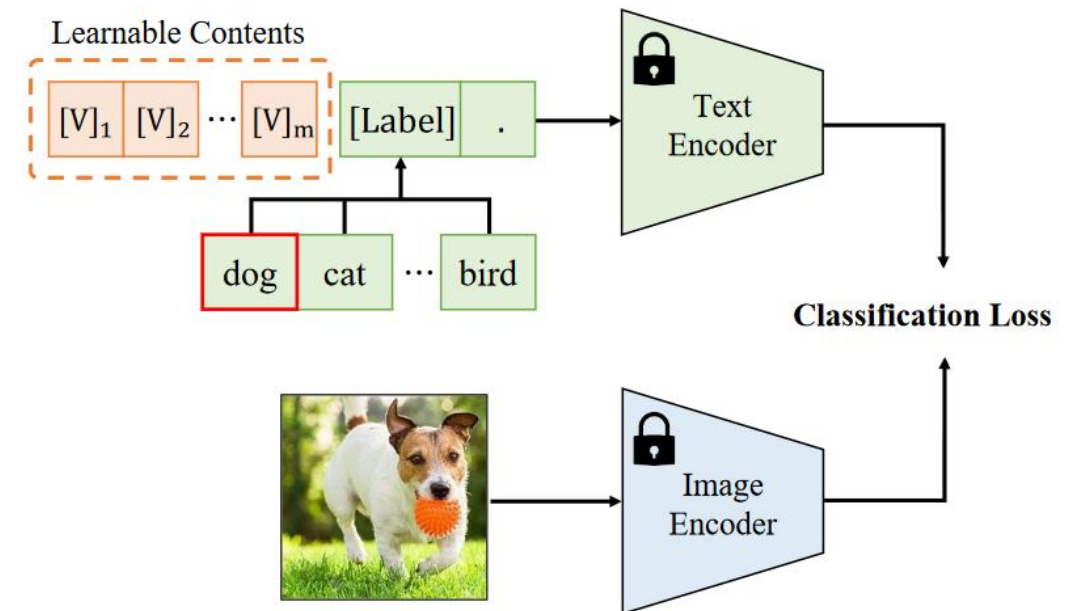
Inspired by NLP, many prompt learning methods have been developed for VLMs to adapt them to downstream tasks by optimizing prompts instead of fine-tuning the entire model.

These methods learn effective and efficient text prompts using only a few labeled samples per class.

Instead of relying on a fixed prompt template such as "a photo of a {}", the surrounding textual context is made learnable.

Examples:

- [CoOp](#): Learning to prompt for vision-language models;
- [CoCoOp](#): Conditional Prompt Learning for Vision-Language Models;
-



Adapters

Adapters augment a pretrained model by adding small trainable modules while keeping the original parameters frozen. These are inserted inside Transformer blocks.

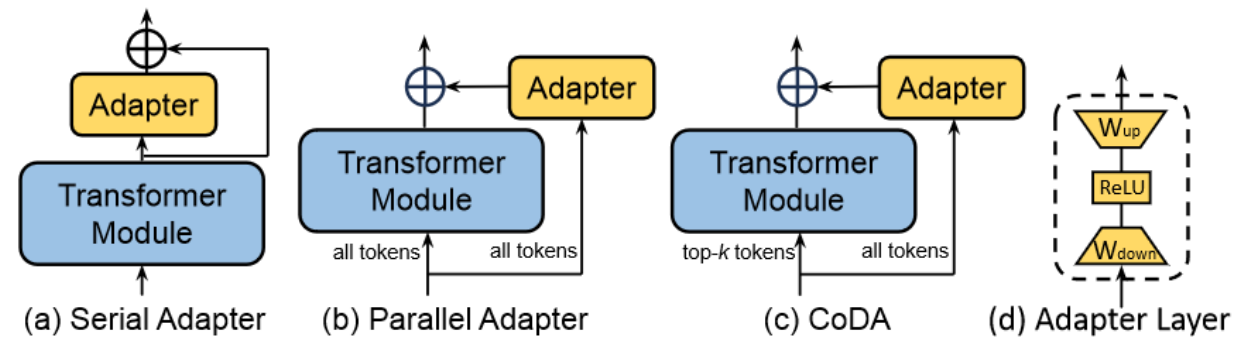
A typical adapter layer consists of:

- a down-projection matrix $W_{\text{down}} \in \mathbb{R}^{d \times r}$
- a non-linear activation $\sigma(\cdot)$
- an up-projection matrix $W_{\text{up}} \in \mathbb{R}^{r \times d}$.

Here, d is the hidden dimension, and r is the bottleneck dimension (a configurable hyperparameter).

Given input h_{in} , the adapter computation (with residual) is:

$$\text{Adapter}(h_{\text{in}}) = \underbrace{W_{\text{up}} \sigma(W_{\text{down}} h_{\text{in}})}_{\text{Adapter output}} + h_{\text{in}}$$



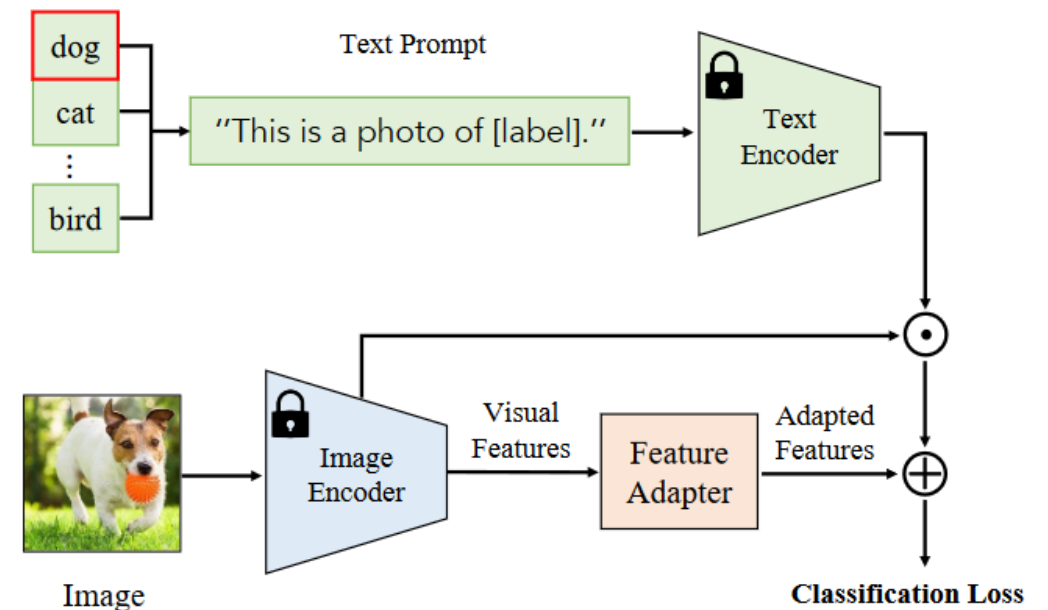
Adapters - Examples

Feature adaptation fine-tunes VLMs by introducing a lightweight adapter to adjust image or text features.

[CLIP-Adapter](#): adds trainable linear layers after CLIP's encoders while keeping the original architecture frozen. To reduce overfitting in few-shot settings, it uses a residual connection that dynamically fuses adapted features with the original ones.

Other Examples:

- [TaskRes](#): Task Residual for Tuning Vision-Language Models;
- [GraphAdapter](#): Tuning Vision-Language Models With Dual Knowledge Graph;



Low-Rank Adaptation

Low-Rank Adaptation (LoRA) reparameterizes model weights by factorizing updates into low-rank matrices, improving parameter efficiency. During inference, these updates can be merged back, leaving inference speed unchanged.

Model weight updates as a low-rank decomposition.
For weight $W \in \mathbb{R}^{d_1 \times d_2}$:

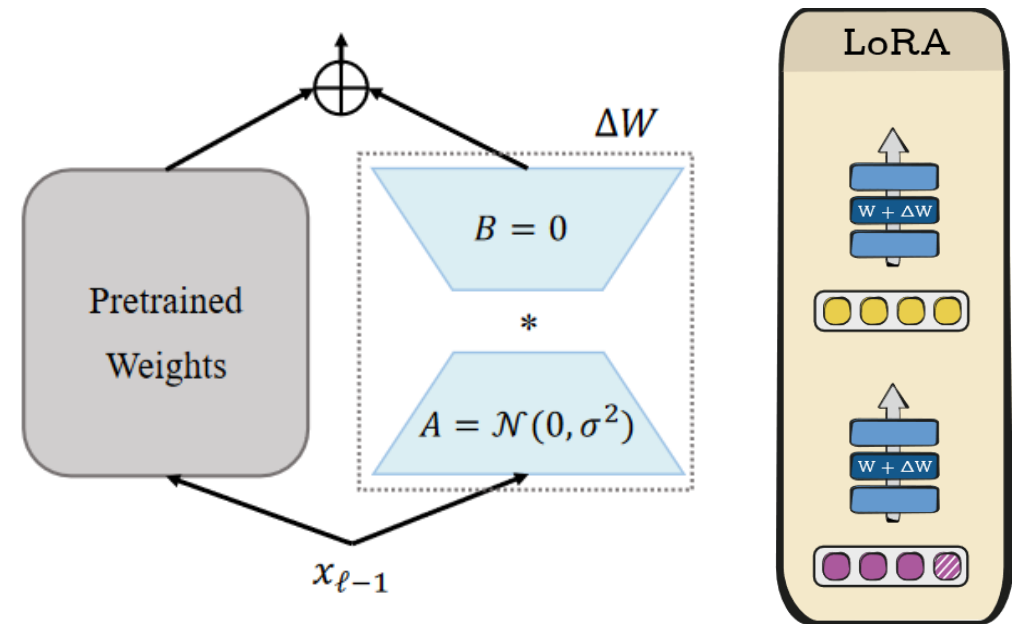
$$x^{(l)} = Wx^{(l-1)} + \gamma \Delta W x^{(l-1)} = Wx^{(l-1)} + \gamma B A x^{(l-1)}$$

where

$$\Delta W = BA, \quad \text{rank}(r) \ll \{d_1, d_2\}$$

$$A \in \mathbb{R}^{r \times d_2}, \quad B \in \mathbb{R}^{d_1 \times r}$$

- A initialized randomly
- B initialized as zeros $\rightarrow$ no change before training



At Inference

$$x^{(l)} = (W + \gamma BA) x^{(l-1)}$$

Example: [LoRA-CLIP](#)

Low-Rank Adaptation (LoRA)

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class LoRALinear(nn.Module):
    def __init__(self, in_features, out_features, r=4, alpha=1.0):
        super().__init__()
        # Freeze original weights
        self.weight = nn.Parameter(torch.randn(out_features, in_features), requires_grad=False)

        # LoRA adapters (low-rank decomposition)
        self.A = nn.Parameter(torch.randn(r, in_features))
        self.B = nn.Parameter(torch.zeros(out_features, r))
        self.scaling = alpha / r

    def forward(self, x):
        base_out = F.linear(x, self.weight)
        lora_update = F.linear(x, self.B @ self.A) * self.scaling
        return base_out + lora_update
```

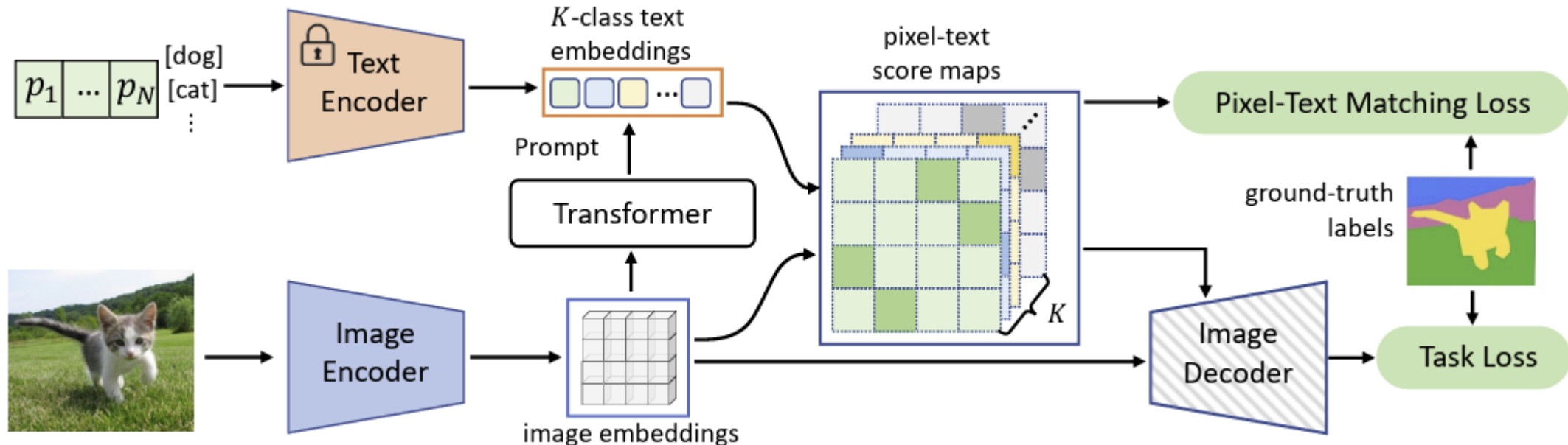
Pre-trained weights:
frozen

Gaussian and zero
initialization of A and B

Transfer Learning Beyond Classification

DenseCLIP (Semantic Segmentation, Object Detection)

- CLIP excels at image-text representation. But transferring to dense prediction tasks (segmentation, detection) is underexplored.
- Convert image-text matching $\rightarrow$ pixel-text matching.
- Use pixel-text score maps to guide dense prediction.
- Apply context-aware prompts from image features to language model.



LINKSFOUNDATION.COM



CONTACTS

Federico D'Asaro

Team Leader

1. federico.dasaro@linksfoundation.com
2. federico.dasaro@polito.it

FONDAZIONE LINKS
Via Pier Carlo Boggio 61 | 10138 Torino
P. +39 011 22 76 150
LINKSFOUNDATION.COM